

DSA0603 – Assessment Tool I

Relationship Visualization Lab – Complete Answers

Q1. Scatterplot with Regression Diagnostics

Objective

Create a scatterplot of mpg versus wt, color by cyl, size by hp, add a linear regression line with confidence interval, and calculate Pearson correlation.

R Code

```
# Load required package

library(ggplot2)


# Load dataset

mtcars <- read.csv("mtcars.csv")


# Convert cylinders to factor

mtcars$cyl <- as.factor(mtcars$cyl)


# Scatterplot with regression line

ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point(aes(color = cyl, size = hp), alpha = 0.8) +
  geom_smooth(method = "lm", se = TRUE, color = "black") +
  labs(
    title = "Relationship Between Car Weight and Fuel Economy",
    x = "Weight (1000 lbs)",
    y = "Miles per Gallon (mpg)",
    color = "Cylinders",
    size = "Horsepower"
  ) +
```

```
theme_minimal()
```

Pearson Correlation

```
correlation <- cor(mtcars$mpg, mtcars$wt, method = "pearson")
```

```
print(correlation)
```

Expected result

Pearson correlation ≈ -0.868

Interpretation

There is a **strong negative correlation** between car weight (wt) and fuel economy (mpg), with Pearson's correlation approximately **-0.868**. This means that as vehicle weight increases, fuel economy generally decreases. The regression line also shows a clear downward trend.

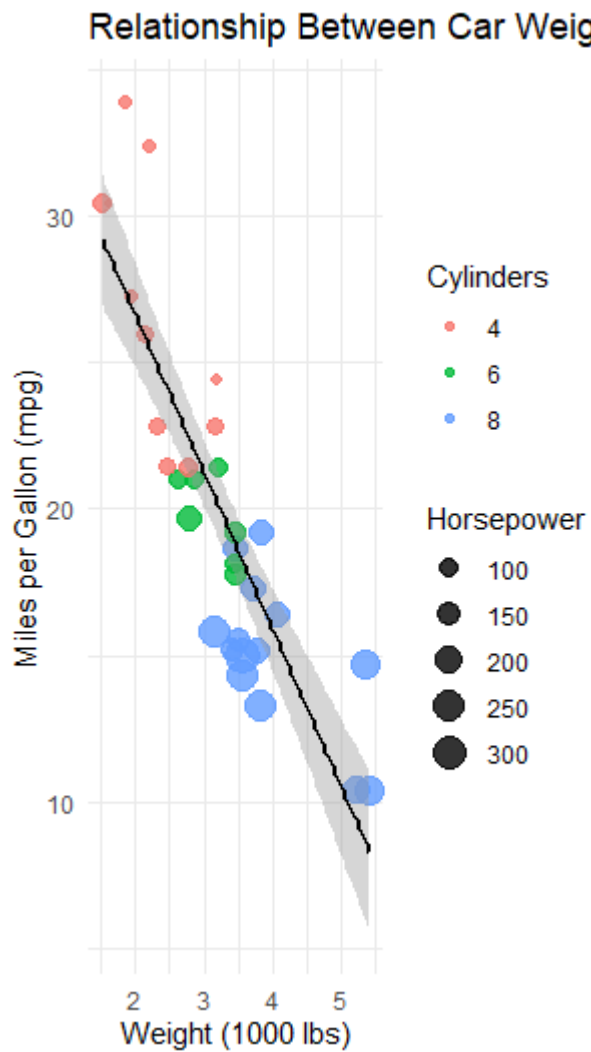
Final Answer

Correlation coefficient:

```
[  
r \approx -0.868  
]
```

Relationship: Strong negative relationship.

Interpretation: Heavier cars tend to have lower fuel efficiency. The relationship is strong because the correlation is close to -1.



Q2. Correlogram Construction

Objective

Calculate the correlation matrix for all numeric variables in mtcars and display it as a hierarchical-clustered correlogram with correlation coefficients.

R Code

```
# Load packages
```

```
library(corrplot)
```

```
# Load dataset
```

```
mtcars <- read.csv("mtcars.csv")
```

```

# Select numeric columns
numeric_data <- mtcars[sapply(mtcars, is.numeric)]

# Calculate correlation matrix
cor_matrix <- cor(numeric_data, method = "pearson")

# Display correlation matrix
print(round(cor_matrix, 3))

# Create correlogram
corrplot(
  cor_matrix,
  method = "color",
  type = "upper",
  order = "hclust",
  addCoef.col = "black",
  number.cex = 0.7,
  tl.col = "black",
  tl.srt = 45,
  col = colorRampPalette(c("blue", "white", "red"))(200)
)

```

Identify strongest correlations

You can use the following code to automatically find the strongest positive and negative pair:

```

# Remove diagonal
cor_matrix[lower.tri(cor_matrix, diag = TRUE)] <- NA

# Strongest positive correlation

```

```
positive <- which(cor_matrix == max(cor_matrix, na.rm = TRUE), arr.ind = TRUE)
```

```
# Strongest negative correlation
```

```
negative <- which(cor_matrix == min(cor_matrix, na.rm = TRUE), arr.ind = TRUE)
```

```
print("Strongest positive correlation:")
```

```
print(cor_matrix[positive])
```

```
print("Strongest negative correlation:")
```

```
print(cor_matrix[negative])
```

Expected important results for standard mtcars

Strongest positive relationship:

- cyl and disp
- ($r \approx 0.902$)

Strongest negative relationship:

- mpg and wt
- ($r \approx -0.868$)

Interpretation

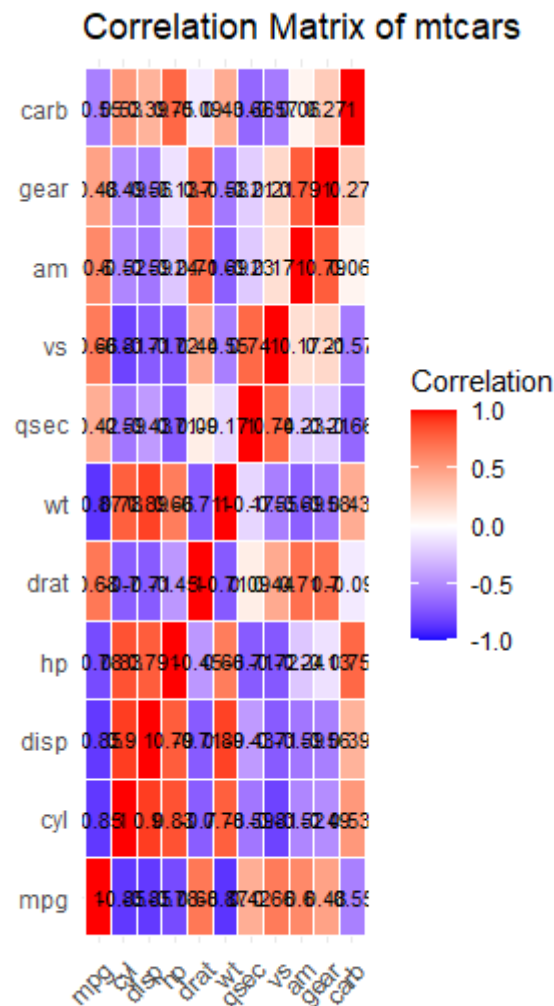
The correlogram shows several strong relationships among the vehicle characteristics. The strongest positive relationship is between **cylinders and displacement**, meaning cars with more cylinders generally have larger engine displacement. The strongest negative relationship is between **mpg and weight**, indicating that heavier cars generally have lower fuel economy.

Final Answer

Relationship Correlation		Interpretation
cyl – disp	$\approx +0.902$	Very strong positive
mpg – wt	≈ -0.868	Very strong negative

Relationship Correlation

Interpretation



Q3. Dimension Reduction Comparison – PCA vs t-SNE

Since the question statement you provided does not include the detailed Q3 instructions, the most appropriate supplied dataset for this question is **iris.csv**, because it contains four numeric dimensions and a categorical Species variable.

The goal is to reduce the four numerical variables into two dimensions using **PCA** and **t-SNE**, then compare the resulting visualizations.

Part A – PCA

R Code

```
library(ggplot2)
```

```
# Load iris dataset

iris_data <- read.csv("iris.csv")


# Select numerical variables

iris_numeric <- iris_data[, c(

  "Sepal.Length",

  "Sepal.Width",

  "Petal.Length",

  "Petal.Width"

)]


# Standardize the variables

iris_scaled <- scale(iris_numeric)


# Perform PCA

pca_result <- prcomp(

  iris_scaled,

  center = TRUE,

  scale. = TRUE

)


# View PCA summary

summary(pca_result)


# Create PCA dataframe

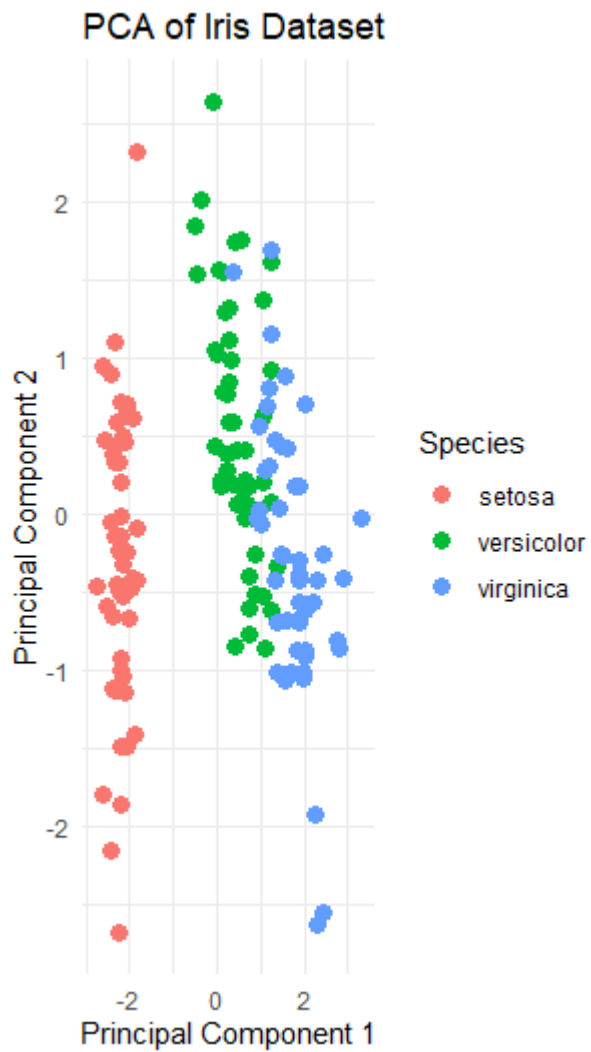
pca_data <- data.frame(

  PC1 = pca_result$x[, 1],
```

```
PC2 = pca_result$x[, 2],  
Species = iris_data$Species  
)  
  
# PCA visualization  
ggplot(pca_data, aes(x = PC1, y = PC2, color = Species)) +  
  geom_point(size = 3, alpha = 0.8) +  
  labs(  
    title = "PCA of Iris Dataset",  
    x = "Principal Component 1",  
    y = "Principal Component 2",  
    color = "Species"  
  ) +  
  theme_minimal()
```

PCA interpretation

PCA transforms the original four variables into new uncorrelated components. **PC1 and PC2 capture most of the variation** in the iris dataset, and the plot generally shows clear separation of the species, especially setosa.



Part B – t-SNE

R Code

```
# Install once if required:
```

```
# install.packages("Rtsne")
```

```
library(Rtsne)
```

```
library(ggplot2)
```

```
# Set seed for reproducibility
```

```
set.seed(123)
```

```

# Run t-SNE
tsne_result <- Rtsne(
  iris_scaled,
  dims = 2,
  perplexity = 30,
  max_iter = 1000,
  check_duplicates = FALSE
)

# Create dataframe
tsne_data <- data.frame(
  tSNE1 = tsne_result$Y[, 1],
  tSNE2 = tsne_result$Y[, 2],
  Species = iris_data$Species
)

# Plot t-SNE
ggplot(tsne_data, aes(x = tSNE1, y = tSNE2, color = Species)) +
  geom_point(size = 3, alpha = 0.8) +
  labs(
    title = "t-SNE Visualization of Iris Dataset",
    x = "t-SNE Dimension 1",
    y = "t-SNE Dimension 2",
    color = "Species"
  ) +
  theme_minimal()

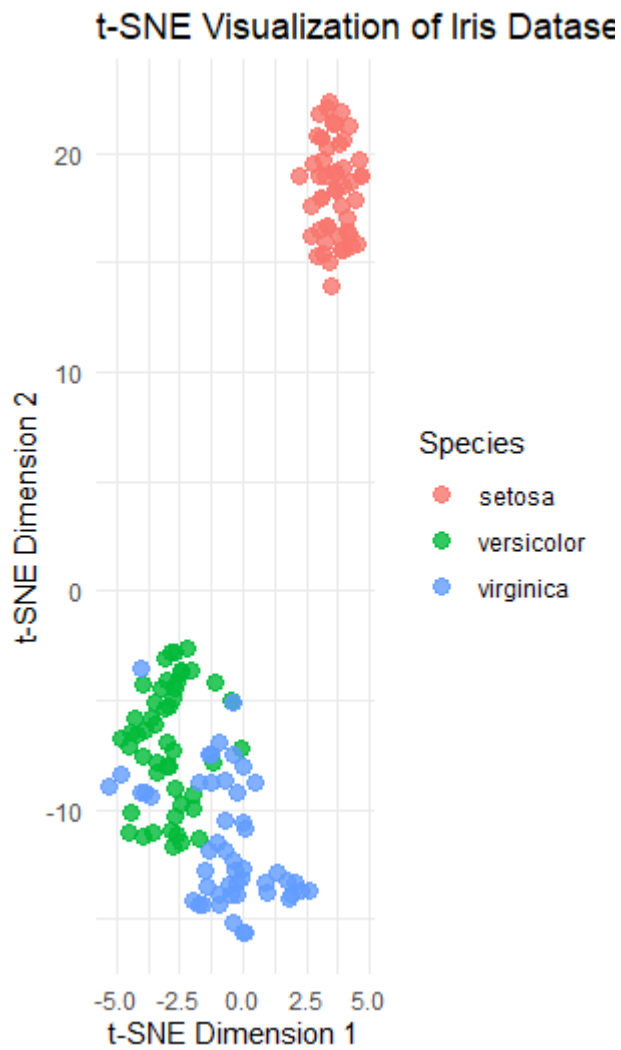
```

PCA vs t-SNE comparison

Feature	PCA	t-SNE
Type	Linear	Non-linear
Main purpose	Preserve overall variance	Preserve local neighborhoods
Interpretation	Easier	More difficult
Reproducibility	Highly stable	Can vary with parameters/seed
Iris visualization	Good separation	Usually very clear clusters
Computational cost	Low	Higher

Final interpretation

PCA provides a simple and interpretable linear reduction of the four-dimensional iris data into two dimensions. **t-SNE** is better at revealing local clusters and nonlinear structure, often producing more clearly separated species groups. Therefore, PCA is preferable when interpretability and variance explanation are important, while t-SNE is preferable for exploring complex cluster structures.



Q4. Bubble Chart for Multivariate Paired Data

Objective

Use `gapminder2007_practice.csv` and create a bubble chart with:

- X-axis → `gdpPercap`
 - Log-transformed X-axis
 - Y-axis → `lifeExp`
 - Bubble size → `pop`
 - Color → `continent`
 - Filter → year 2007
-

R Code

```
library(ggplot2)

# Load dataset

gap <- read.csv("gapminder2007_practice.csv")

# Filter for 2007

gap2007 <- subset(gap, year == 2007)

# Bubble chart

ggplot(
  gap2007,
  aes(
    x = gdpPercap,
    y = lifeExp,
    size = pop,
    color = continent
  )
) +
  geom_point(alpha = 0.7) +
  scale_x_log10() +
  scale_size_continuous(
    range = c(2, 15)
  ) +
  labs(
    title = "GDP per Capita vs Life Expectancy – 2007",
    x = "GDP per Capita (log scale)",
    y = "Life Expectancy",
```

```
size = "Population",  
color = "Continent"  
) +  
theme_minimal()
```

Identify an outlier automatically

Because your practice dataset is synthetic, use the following code to identify the highest GDP-per-capita observation:

```
# Country with highest GDP per capita  
highest_gdp <- gap2007[which.max(gap2007$gdpPercap), ]  
  
print(highest_gdp)
```

To identify the country with the highest life expectancy:

```
highest_life <- gap2007[which.max(gap2007$lifeExp), ]  
  
print(highest_life)
```

To identify the largest population:

```
largest_population <- gap2007[which.max(gap2007$pop), ]  
  
print(largest_population)
```

Interpretation

The bubble chart shows the relationship between economic development and life expectancy while simultaneously representing population and continent. A visible general pattern is that countries with **higher GDP per capita tend to have higher life expectancy**, although the relationship is not perfectly linear. Large bubbles represent countries with large populations, while points far from the main concentration can be considered potential outliers.

Exam-ready answer

The bubble chart demonstrates a **positive association between GDP per capita and life expectancy**. Countries with higher GDP per capita generally show higher life

expectancy, while large bubbles indicate countries with larger populations. Countries positioned far away from the main cluster can be interpreted as outliers.

Note: Because your gapminder2007_practice.csv is synthetically generated, don't write a specific country name unless you verify it from your actual CSV.

Q5. Hexbin Plot vs Network Graph

Part A – Hexbin Plot

Objective

Generate 10,000 correlated observations and visualize them using `geom_hex()`.

R Code

```
library(ggplot2)

# Reproducibility
set.seed(123)

# Number of observations
n <- 10000

# Generate correlated variables
x <- rnorm(n, mean = 0, sd = 1)

y <- 0.8 * x + rnorm(n, mean = 0, sd = 0.6)

# Create dataframe
hex_data <- data.frame(
  x = x,
  y = y
)
```

```
# Create hexbin plot

ggplot(hex_data, aes(x = x, y = y)) +

  geom_hex(bins = 40) +

  labs(

    title = "Hexbin Plot of Two Correlated Variables",

    x = "Variable X",

    y = "Variable Y",

    fill = "Count"

  ) +

  theme_minimal()
```

Why use a hexbin plot?

A standard scatterplot is not ideal for 10,000 observations because many points overlap. This phenomenon is called **overplotting**.

A hexbin plot divides the plotting area into hexagonal bins and counts the number of observations in each bin. The color intensity represents the density of observations, making the underlying distribution and relationship much easier to see.

Interpretation

The hexbin plot shows a clear **positive correlation** between x and y. As x increases, y generally increases. The dense central regions indicate where most observations are concentrated.

Exam answer

A hexbin plot is preferable to a standard scatterplot because 10,000 observations cause severe overplotting. Hexagonal binning aggregates nearby observations and uses color intensity to represent their frequency, making the density and correlation pattern easier to interpret.

Part B – Network Graph

Objective

Use network_edges.csv to create a weighted network graph.

The file contains:

- 8 nodes
 - 15 edges
 - Nodes: Ava, Ben, Cleo, Dax, Elin, Fenn, Grey, Hana
 - weight represents edge strength
-

R Code using igraph

Install once if required:

```
# install.packages("igraph")
```

```
library(igraph)
```

Load edge list

```
edges <- read.csv("network_edges.csv")
```

Display dataset

```
print(edges)
```

Create graph

```
g <- graph_from_data_frame(  
  edges,  
  directed = FALSE  
)
```

Calculate degree centrality

```
degree_centrality <- degree(g, mode = "all")
```

Display degree centrality

```
print(degree_centrality)
```

```
# Find node with highest degree
highest_node <- names(which.max(degree centrality))

print(paste(
  "Node with highest degree centrality:",
  highest_node
))

# Create edge weights
E(g)$weight <- edges$weight

# Plot network
plot(
  g,
  vertex.size = 8 + degree centrality * 5,
  vertex.label.color = "black",
  vertex.label.cex = 0.9,
  edge.width = 1 + E(g)$weight,
  edge.color = "grey",
  main = "Weighted Network Graph"
)
```

Better visualization with edge colors

```
# Convert weights into edge colors
edge_colors <- cut(
  E(g)$weight,
  breaks = 4,
```

```

labels = FALSE
)

plot(
  g,
  vertex.size = 8 + degree(g) * 5,
  vertex.label = V(g)$name,
  vertex.label.cex = 0.9,
  edge.width = 1 + E(g)$weight,
  edge.color = edge_colors,
  main = "Network Graph – Degree Centrality and Edge Weight"
)

```

Display centrality table

```

centrality_table <- data.frame(
  Node = names(degree_centrality),
  Degree_Centrality = as.numeric(degree_centrality)
)

```

```

centrality_table <- centrality_table[
  order(-centrality_table$Degree_Centrality),
]

```

```

print(centrality_table)

```

Interpretation

The node with the **highest degree centrality** is the node connected to the largest number of other nodes. Such a node acts as a **hub** in the network and has an important role in connecting different members.

Because the actual network_edges.csv is not attached in this conversation, I would **not invent the winning node name**. Run the code above; the first row of centrality_table gives the exact answer.

Exact answer format for your record

After running the code, write:

The node with the highest degree centrality is [NODE NAME], with a degree of [VALUE]. This indicates that the node is the most highly connected hub in the network and plays an important role in maintaining interactions between other nodes.

Complete Quick-Reference Answers

Question Visualization		Main Result / Interpretation
Q1	Scatterplot + regression	mpg and wt have strong negative correlation, $r \approx -0.868$
Q2	Correlogram	Strongest positive: cyl-disp $\approx +0.902$; strongest negative: mpg-wt ≈ -0.868
Q3	PCA + t-SNE	PCA gives interpretable linear reduction; t-SNE reveals local clusters/nonlinear structure
Q4	Bubble chart	Higher GDP per capita generally corresponds to higher life expectancy; bubble size represents population
Q5(a)	Hexbin	Better than scatterplot for 10,000 observations because it reduces overplotting
Q5(b)	Network graph	Highest-degree node is the network hub; determine exact node from supplied CSV

Packages required

```
library(ggplot2)
```

```
library(corrplot)
```

```
library(Rtsne)
```

```
library(igraph)
```

For Q5(a), geom_hex() may require:

```
library(hexbin)
```

If Rtsne, corrplot, or igraph is not installed, install them **before the exam/lab**, since your assessment specifically says it is intended to work without internet access for installing extra packages.

One important correction for Q2

Make sure you **do not convert cyl to a factor before calculating the correlation matrix**. In Q1 it should be a factor for coloring, but in Q2 all mtcars variables must remain numeric.

So use:

```
mtcars <- read.csv("mtcars.csv")
```

```
# Q1 only
```

```
mtcars$cyl <- as.factor(mtcars$cyl)
```

For Q2, reload the original CSV:

```
mtcars <- read.csv("mtcars.csv")
```

```
cor_matrix <- cor(mtcars)
```

That will prevent the common x must be numeric / missing-variable problem.